Desafio DevOps - Fase 6

Projeto Cidades ESG Inteligentes | EcoGestor API

Giovanni Luiz do Rego - RM 563071

API em C# para gestão de resíduos e reciclagem: unidades, pontos de coleta, coletas, destinações, alertas e indicadores ESG.

Resultado da execução

Pipeline GitHub Actions concluído com sucesso: build, 12 testes aprovados e implantação sequencial em staging e produção simulada. Os dois ambientes retornaram status ready e duas unidades cadastradas.

Repositório do projeto

Repositório público: GiovanniLuiz/EcoGestor-DevOps

As evidências de build, testes, deploy e funcionamento estão nas páginas seguintes. O ZIP inclui os arquivos em docs/evidencias e o índice em docs/EVIDENCIAS.md.

Arquitetura e execução

Isolamento dos ambientes

GitHub Actions CI → imagem aprovada por commit → runner Windows local → staging → produção simulada. Cada projeto Compose contém uma API ASP.NET Core e um banco SQL Server, conectados pela rede backend do respectivo projeto.

Local: porta 8080, projeto ecogestor-local.
Staging: porta 8081, projeto ecogestor-staging.
Produção simulada: porta 8082, projeto ecogestor-production.

Cada banco usa seu próprio volume sql-data prefixado pelo projeto. A porta do banco não é publicada no host. Os healthchecks condicionam a inicialização da API à prontidão do SQL Server.

Dockerfile e tecnologias

Build em estágio SDK .NET 8; publicação em Release; execução em imagem ASP.NET 8 como usuário app. O curl atende ao healthcheck. A tag da imagem corresponde ao SHA do commit; a mesma imagem é promovida entre os ambientes.

C#, ASP.NET Core 8, Entity Framework Core 8, SQL Server 2022 Developer, JWT, Swagger/OpenAPI, xUnit, Docker Compose, GitHub Actions e PowerShell.

Comandos na raiz do projeto

./scripts/New-LocalEnvironment.ps1

docker compose -p ecogestor-local up -d --build --wait --wait-timeout 300

docker compose -p ecogestor-local ps

docker compose -p ecogestor-local stop

O primeiro comando gera .env com credenciais locais; preserve esse arquivo e não o publique. Acesse localhost:8080/swagger ou /health/ready. O README contém o Dockerfile completo, comandos adicionais e instruções de autenticação.

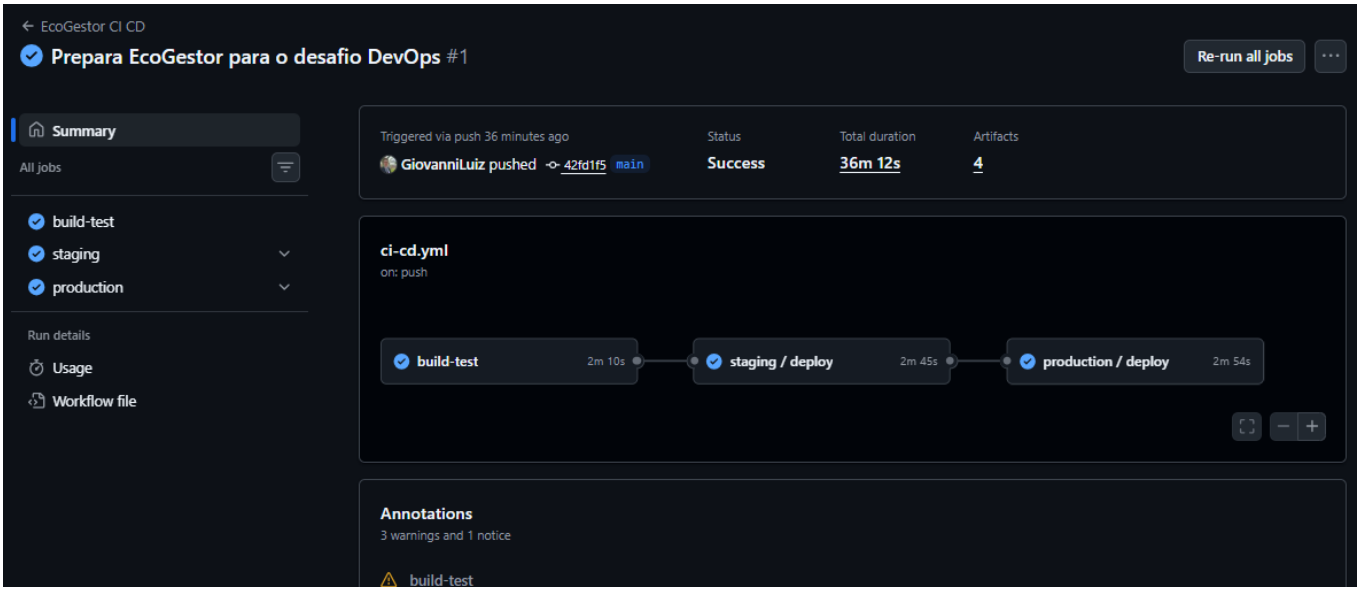
Pipeline CI/CD

Pull requests para main executam restauração, build, testes, imagem Docker e integração com SQL Server. Push ou execução manual em main promovem a imagem aprovada para staging e, após sucesso, para produção.

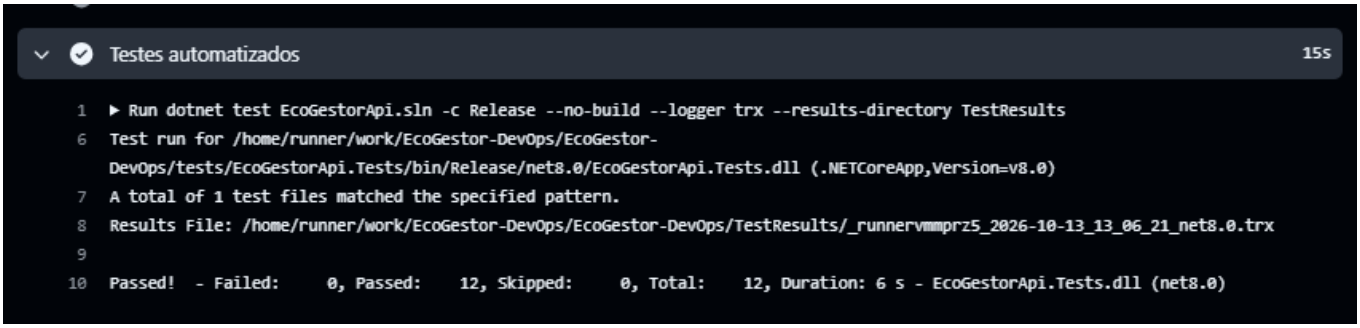
O runner ecogestor-windows usa as labels self-hosted, Windows, X64 e ecogestor. Foi iniciado interativamente. Cada environment contém DB_PASSWORD, JWT_KEY e ADMIN_PASSWORD, sem valores expostos na documentação.

Cada deploy verifica prontidão, ambiente, versão, unidades, login e relatório protegido. Falhas bloqueiam jobs dependentes. Não há rollback automático. Os dados permanecem em volumes separados.

E04 | Pipeline concluído: build, staging e produção



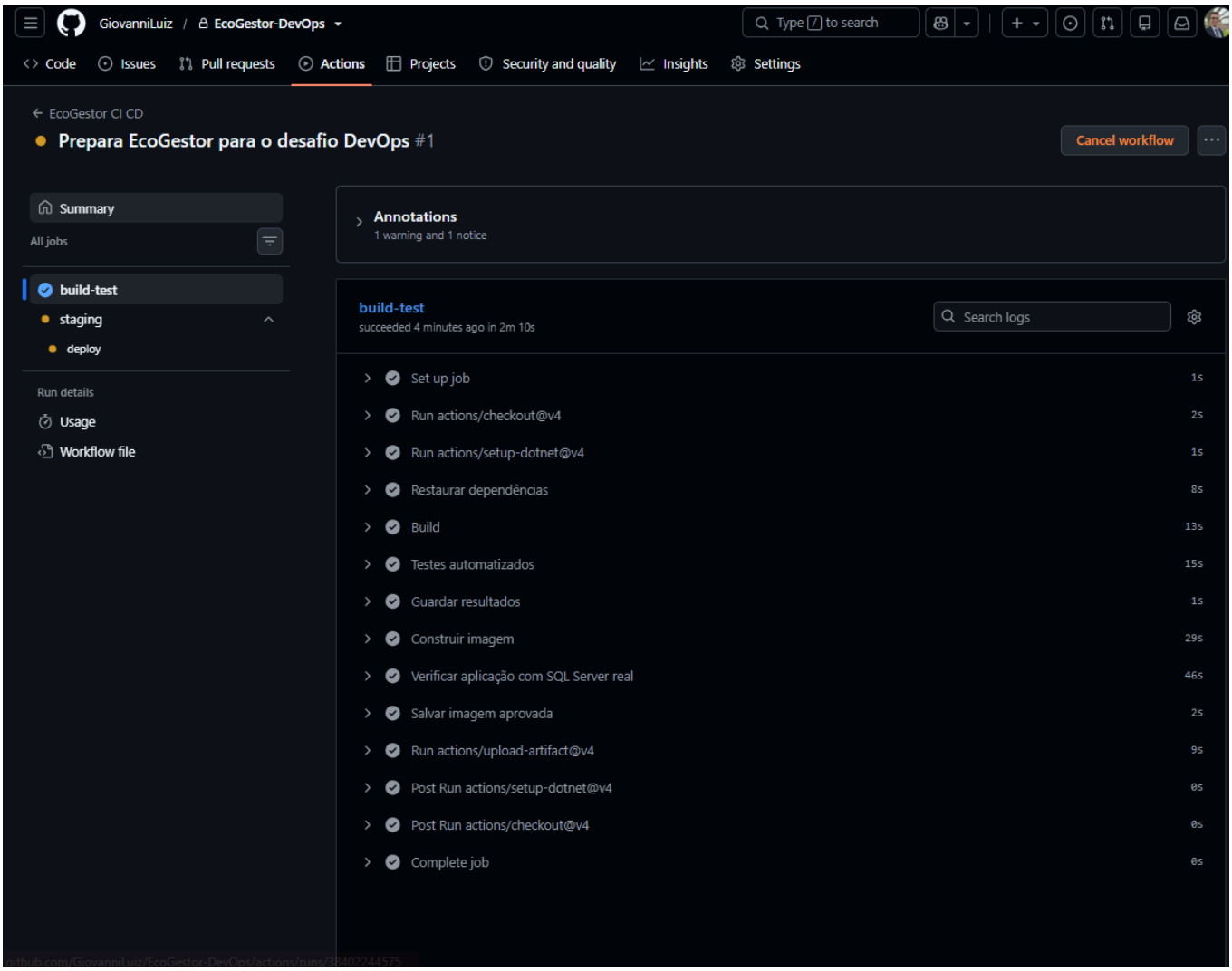
E03 | Testes automatizados: 12 aprovados, zero falhas



Build-test: 2min10s; staging: 2min45s; produção: 2min54s. O tempo total de 36min12s inclui espera pelo runner. O resumo registra quatro artefatos e avisos, com resultado final Success.

Build e testes no GitHub

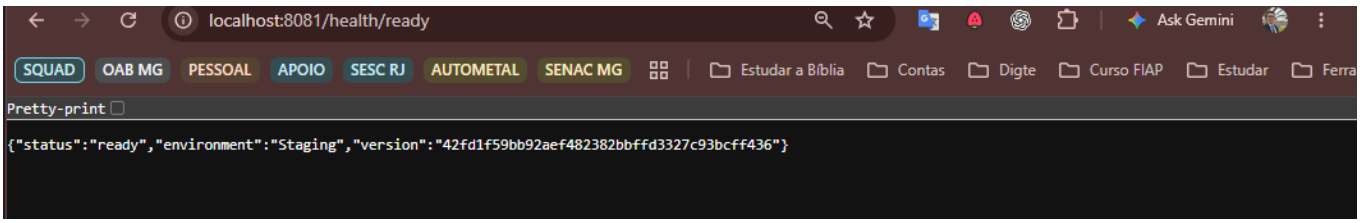
E02 | Etapas de build e integração no GitHub



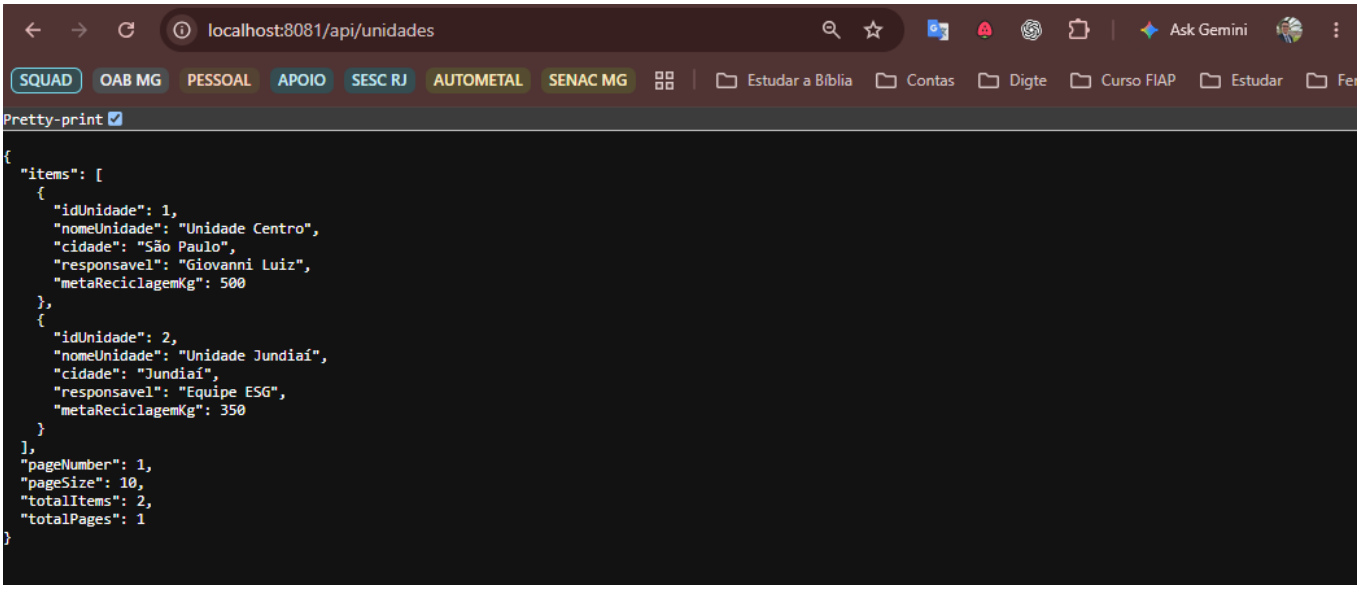
A captura registra as etapas aprovadas do job build-test. A evidência E03 detalha o resultado: 12 testes aprovados, zero falhas e zero ignorados, com duração de 6 segundos. Os testes usam InMemory; a integração do pipeline verifica SQL Server real.

Staging em funcionamento

E05 | Staging: prontidão na porta 8081



E06 | Staging: consulta de duas unidades



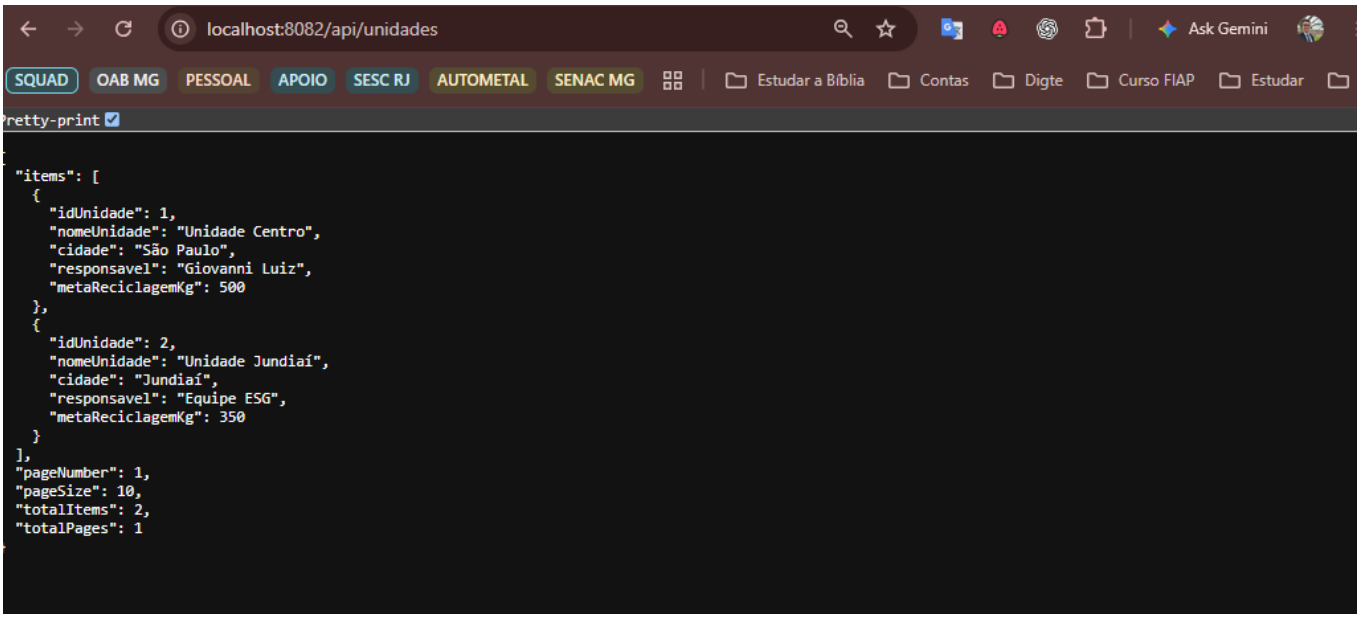
localhost:8081/health/ready retornou ready e ambiente Staging. A consulta /api/unidades retornou Unidade Centro e Unidade Jundiaí, com totalItems igual a 2.

Produção simulada em funcionamento

E07 | Produção simulada: prontidão na porta 8082



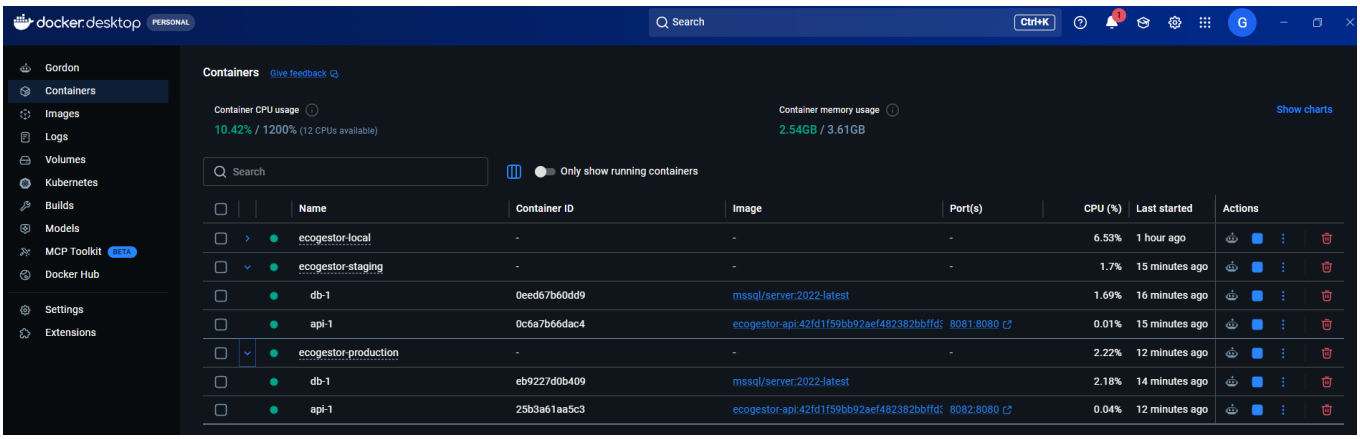
E08 | Produção simulada: consulta de duas unidades



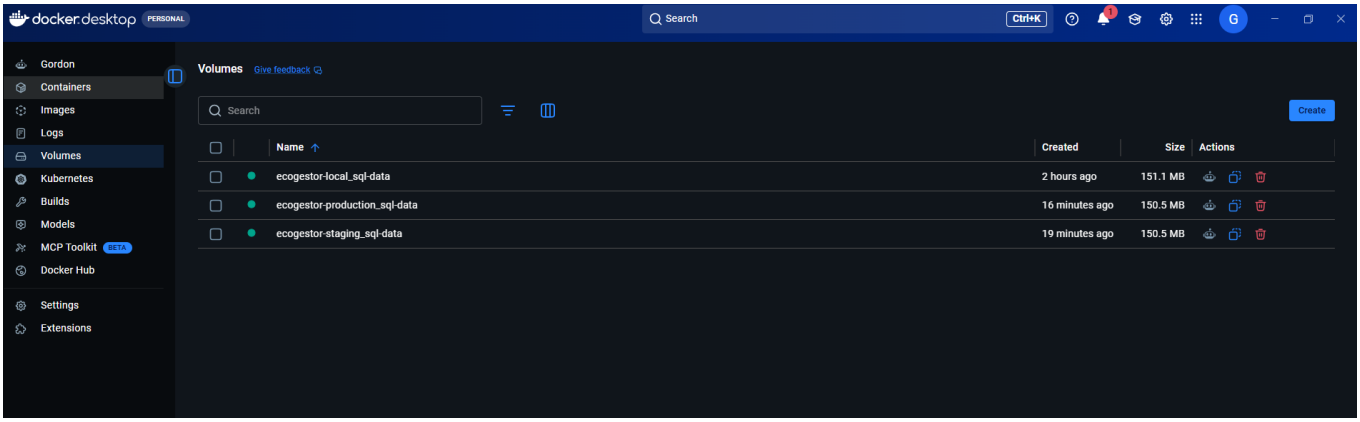
localhost:8082/health/ready retornou ready e ambiente Production, com a mesma versão de staging. /api/unidades retornou os dois registros iniciais.

Containers e persistência

E09 | Containers dos ambientes no Docker Desktop



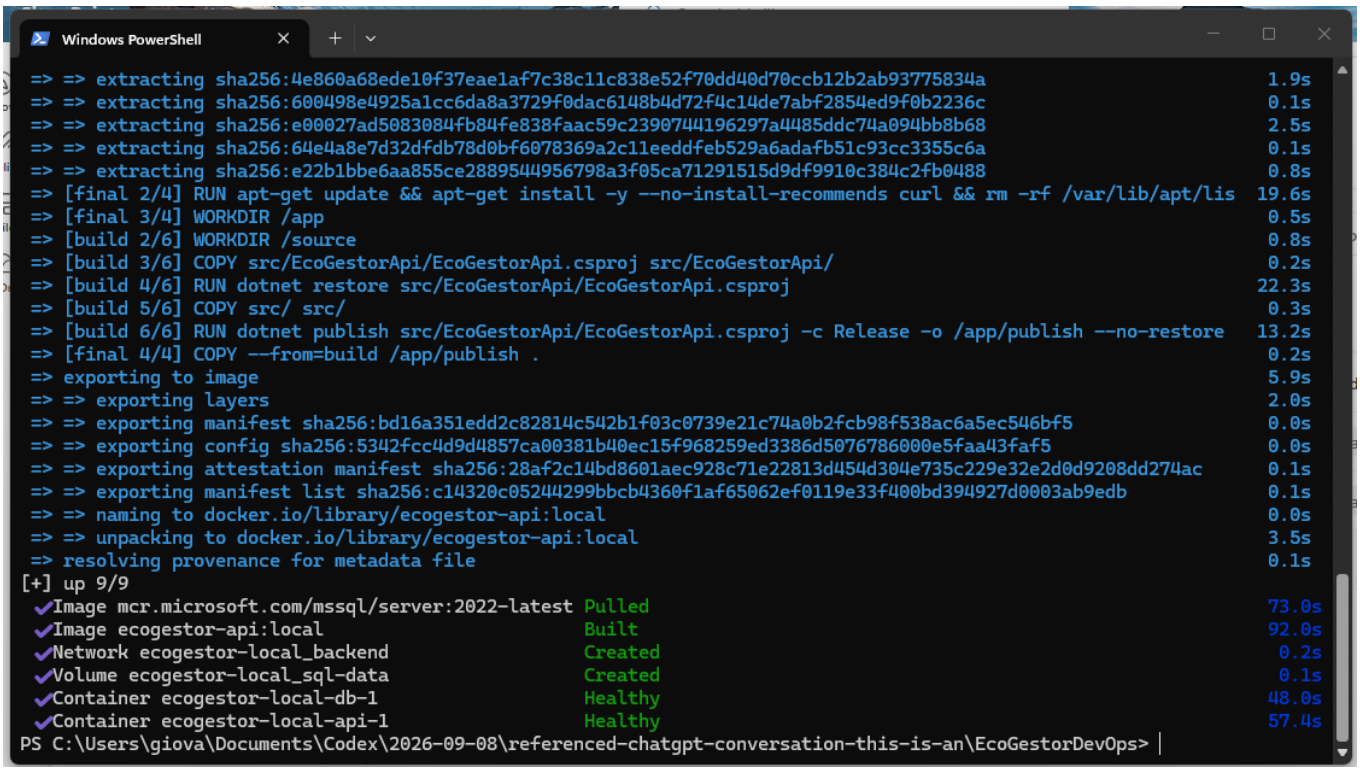
E10 | Volumes separados para local, staging e produção



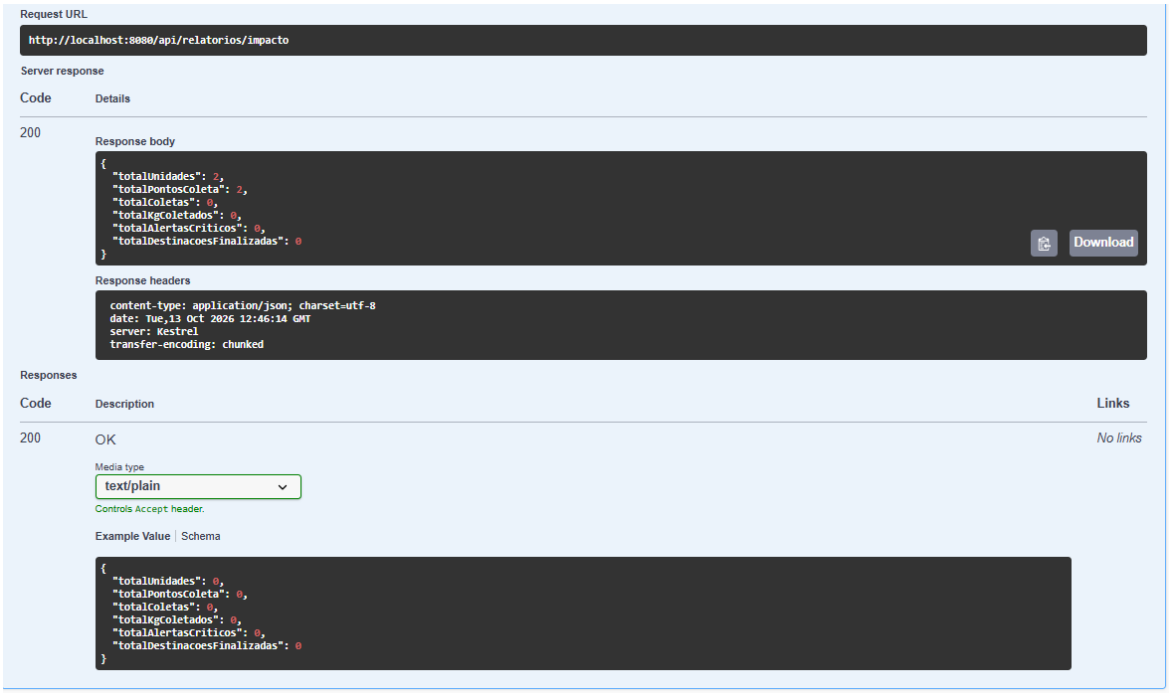
Docker Desktop mostra API e banco em execução nos dois ambientes, com portas 8081 e 8082. Os três volumes distintos correspondem a local, staging e produção. As capturas comprovam a existência dos volumes; um teste de recuperação de backup não faz parte destas evidências.

Verificação local complementar

E01 | Docker local: construção e inicialização



E11 | Relatório protegido: resposta HTTP 200 no teste local



Desafios, soluções e checklist

Desafios resolvidos

Infraestrutura local: instalação de Docker Desktop e ativação do WSL 2 para executar Linux containers. Autenticação Git: login pelo Git Credential Manager após falha de credenciais no primeiro push. Execução dos deploys: registro e ativação do runner Windows com acesso ao Docker Desktop.

Testes: fixtures isoladas com banco InMemory e verificação de geração do SQL de migration. Dados e credenciais: volumes por ambiente e segredos nos environments do GitHub.

Checklist obrigatório - concluído

Concluído - Código-fonte e testes organizados no ZIP.

Concluído - Dockerfile construído e executado.

Concluído - Compose com rede, volumes e variáveis, executado com sucesso.

Concluído - Pipeline com build, testes e deploy em dois ambientes, aprovado no GitHub.

Concluído - README com instruções de execução, tecnologias, Dockerfile e capturas.

Concluído - PDF final com evidências reais de build, testes e deploy.

Concluído - Staging e produção simulada verificados por prontidão e consulta.

Concluído - Pacote sem .env real, .git, bin, obj ou imagens Docker temporárias.

Consulta e reprodução

Use README.md para execução, docs/CONFIGURACAO.md para preparar GitHub e runner, docs/VALIDACAO.md para resultados e limites e docs/EVIDENCIAS.md para localizar capturas. O repositório é público e pode ser consultado sem autorização. O ZIP também reúne o código, as configurações e a documentação.